

Practical No. 5

Study and implementation of Express.js

Name - Atharv Vinayak Kulkarni

Batch - S5

PRN - 245109074

Perform following problem statements using Express.js

Problem Statement 1: Basics of Express.js

1) What is Express.js and how does it differ from Node.js?

Ans –

Express.js

- Express.js is a **web framework for Node.js**
- It simplifies building **web servers and APIs**
- Provides features like:
 - Routing
 - Middleware
 - Request handling

Node.js

- Node.js is a **runtime environment**
- It allows JavaScript to run **outside the browser**
- Used for backend development

Difference

Feature	Node.js	Express.js
Type	Runtime	Framework
Level	Low-level	High-level
Routing	Manual	Built-in
Ease	Complex	Easy

2) How do you create a simple Express.js server?

Ans –

Steps:

1. Install express:

```
npm init -y  
npm install express
```

2. Create server file:

```
const express = require('express');  
const app = express();  
app.get('/', (req, res) => {  
  res.send('Hello World');  
})  
app.listen(3000, () => {  
  console.log('Server running on port 3000');  
});
```

3) Explain the concept of routing in Express.js. How do you define routes?

Ans –

Definition:

- Routing means **handling different URLs (endpoints)**

Example:

```
app.get('/about', (req, res) => {  
  res.send('About Page');  
});
```

Types of Routes:

- GET → Fetch data
- POST → Send data
- PUT → Update
- DELETE → Remove

4) What is middleware in Express.js, and how does it work?

Ans -

Definition:

- Middleware is a **function that runs between request and response**

Flow:

Request → Middleware → Response

Example:

```
app.use((req, res, next) => {  
  console.log('Request received');  
  next();  
});
```

5) How do you create and use custom middleware in an Express.js application?

Ans -

Definition:

- Custom middleware is a **user-defined function** used in request-response cycle.

Steps:

1. Create function (req, res, next)
2. Write logic
3. Call next()
4. Use with app.use()

Example:

```
const logger = (req, res, next) => {  
  console.log(req.url);  
  next();  
};  
app.use(logger)
```

Key Points:

- Runs before route handler
- next() is compulsory
- Used for logging, authentication

6) What is the difference between application-level middleware and router-level middleware?

Ans -

Type	Description
Application-level	Applied using app.use()
Router-level	Applied using express.Router()

7) What are req and res in Express.js? Give examples of common properties and methods associated with each.

Ans –

req (Request):

- Contains client data
- Common properties:
 - req.params - Route parameters
 - req.query - Query strings
 - req.body - POST data
 - req.url - Current URL

res (Response):

- Sends response to client
- Common methods:
 - res.send() - Send response
 - res.json() - Send JSON
 - res.status() - Set status code
 - res.render() - Render views

Example:

JavaScript

```
app.get('/user', (req, res) => {  
  console.log(req.query.name); // Access query  
  res.send('Hello ' + req.query.name);  
});
```

8) How would you extract query parameters from a URL in an Express.js route?

Ans –

Definition:

- Query parameters come after ? in URL
- Format: /search?name=John&age=25

Example:

JavaScript

```
app.get('/search', (req, res) => {  
  const name = req.query.name;  
  const age = req.query.age;  
  res.send(`Name: ${name}, Age: ${age}`);  
});
```

URL: http://localhost:3000/search?name=John&age=25

9) How does Express.js handle different HTTP methods (GET, POST, PUT, DELETE)?

Ans –

Different Methods:

JavaScript

```
// GET - Retrieve data
```

```
app.get('/data', (req, res) => {  
  res.send('GET request');  
});
```

```
// POST - Create data
```

```
app.post('/data', (req, res) => {  
  res.send('POST request');  
});
```

```
// PUT - Update data
```

```
app.put('/data', (req, res) => {  
  res.send('PUT request');  
});
```

```
// DELETE - Remove data
```

```
app.delete('/data', (req, res) => {  
  res.send('DELETE request');  
});
```

10) What are route parameters in Express.js? How do you use them in a route definition?

Ans –

Definition:

- Dynamic values in URL path
- Defined with : colon

Example:

JavaScript

```
app.get('/user/:id', (req, res) => {  
  const userId = req.params.id;  
  res.send(`User ID: ${userId}`);  
});
```

```
app.get('/post/:id/:title', (req, res) => {  
  res.send(`ID: ${req.params.id}, Title: ${req.params.title}`);  
});
```

URL: http://localhost:3000/user/123

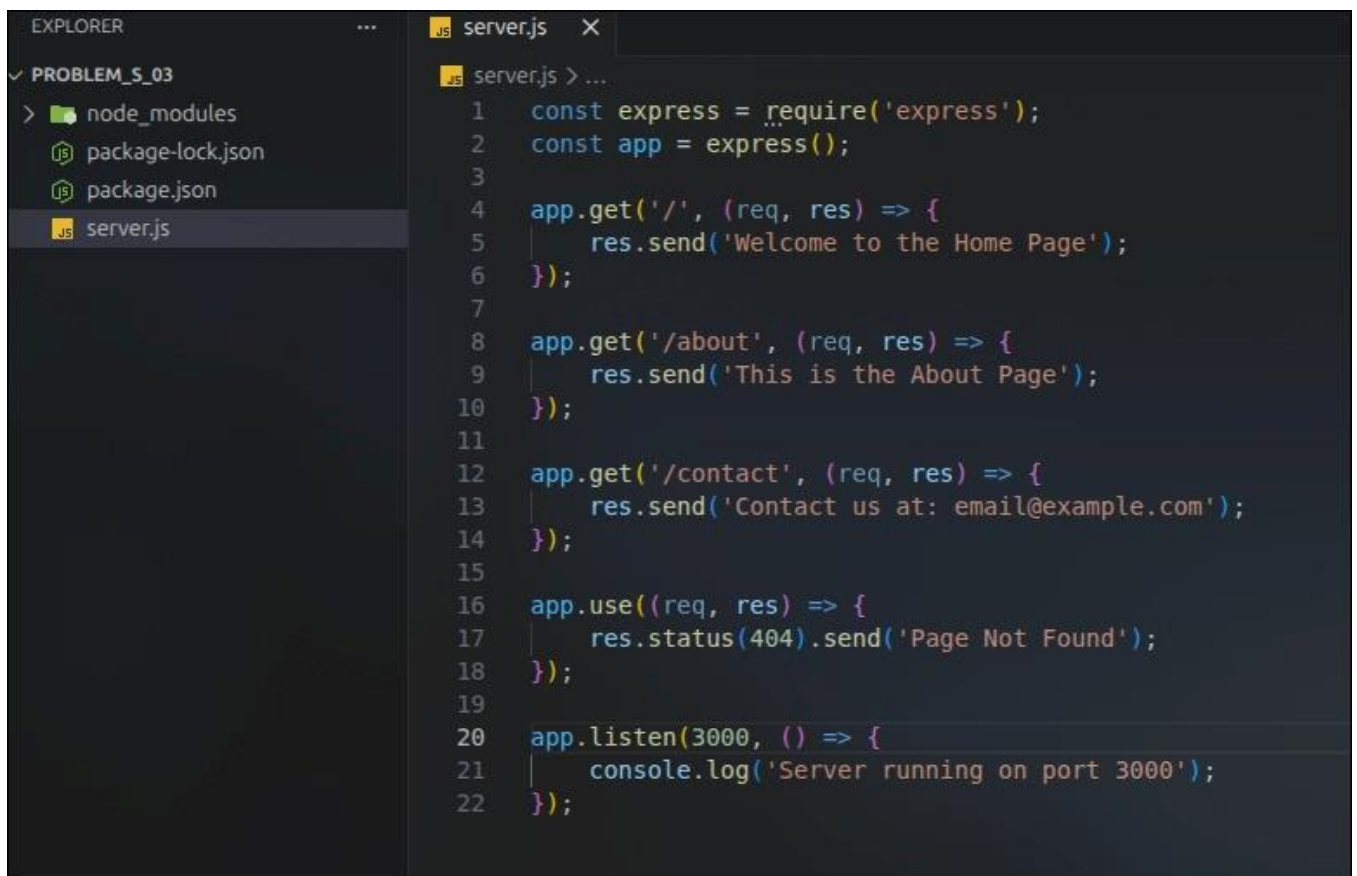
Output: User ID: 123

Problem Statement 2: Basic Web Server with Express JS

Requirements

- Create a basic Express.js server that listens on port 3000.
- Define three routes:
 - GET / - Responds with "Welcome to the Home Page".
 - GET /about - Responds with "This is the About Page".
 - GET /contact - Responds with "Contact us at: email@example.com".
- Include a 404 error handler that displays a "Page Not Found" message for unknown routes.

File: server.js

A screenshot of a code editor (VS Code) showing a file named 'server.js'. The left sidebar shows the 'EXPLORER' view with a project named 'PROBLEM_5_03' containing 'node_modules', 'package-lock.json', 'package.json', and 'server.js'. The main editor area displays the code for 'server.js' with line numbers 1 through 22. The code uses ES6 arrow functions and Express.js to create a web server listening on port 3000 with three GET routes and a 404 error handler.

```
1  const express = require('express');
2  const app = express();
3
4  app.get('/', (req, res) => {
5    res.send('Welcome to the Home Page');
6  });
7
8  app.get('/about', (req, res) => {
9    res.send('This is the About Page');
10 });
11
12 app.get('/contact', (req, res) => {
13   res.send('Contact us at: email@example.com');
14 });
15
16 app.use((req, res) => {
17   res.status(404).send('Page Not Found');
18 });
19
20 app.listen(3000, () => {
21   console.log('Server running on port 3000');
22 });
```

Steps to Run:

Bash

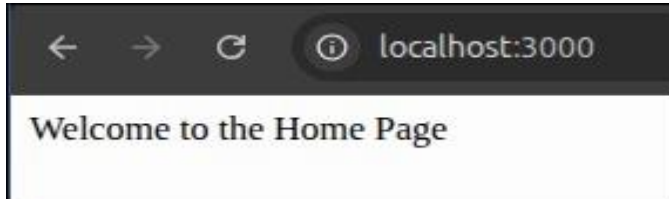
npm init -y

npm install express

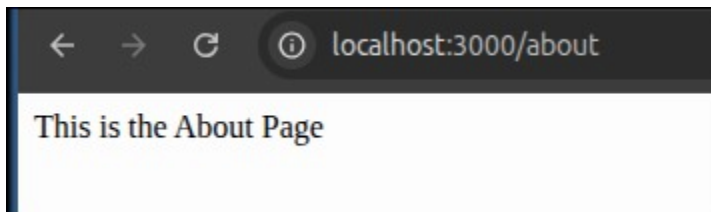
node server.js

Test URLs:

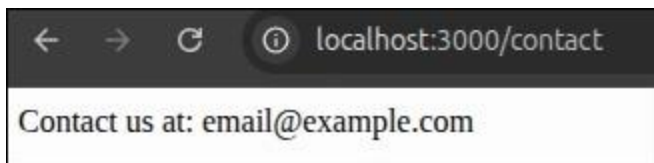
- <http://localhost:3000/> → Welcome to the Home Page



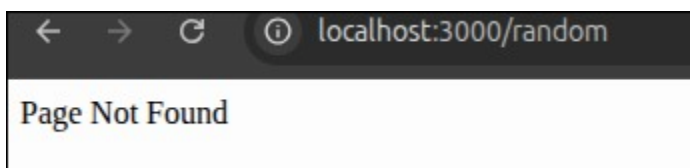
- <http://localhost:3000/about> → This is the About Page



- <http://localhost:3000/contact> → Contact us at: email@example.com



- <http://localhost:3000/random> → Page Not Found

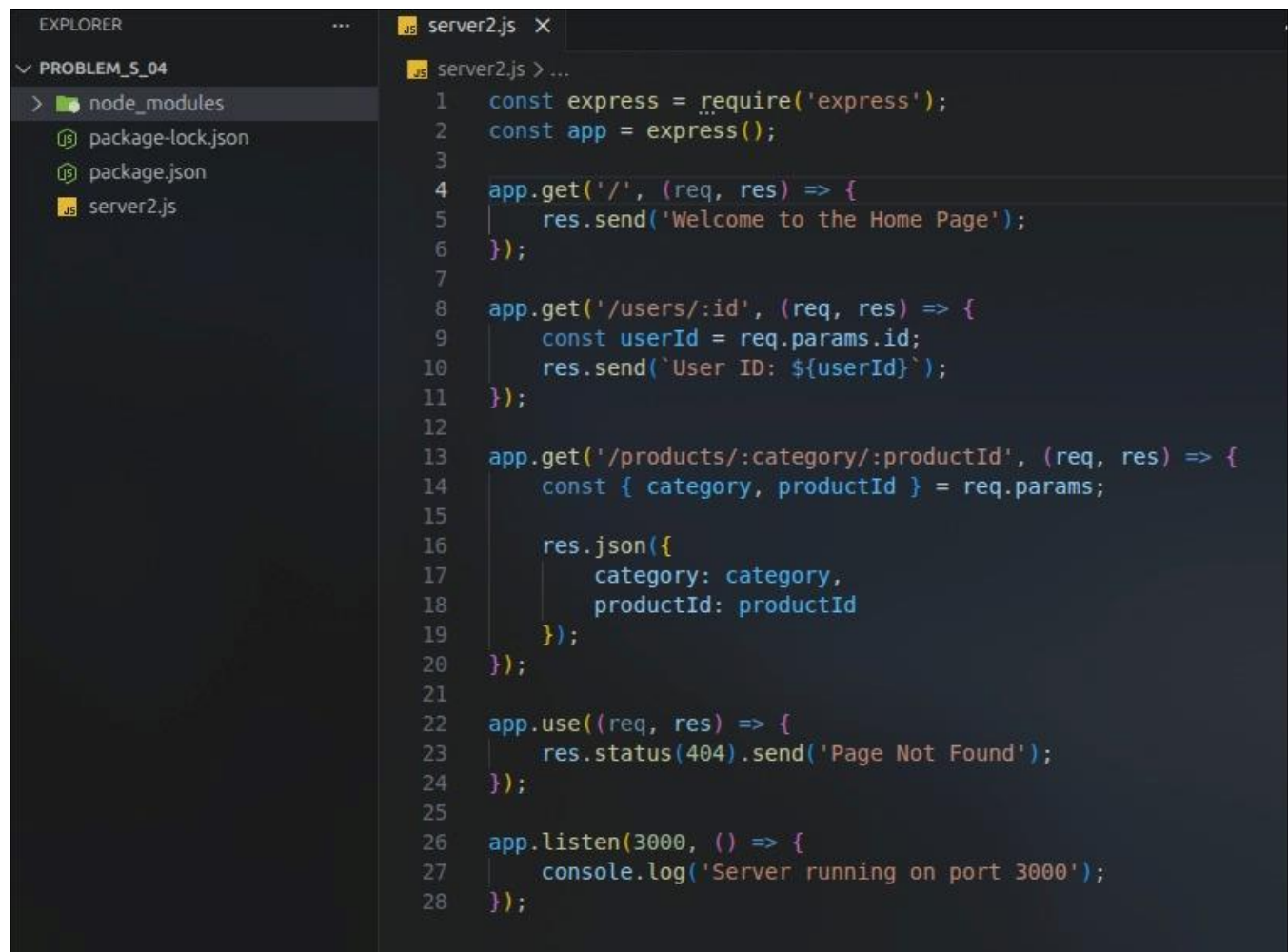


Problem Statement 3: Dynamic Route Parameters

Requirements

- Modify the previous server to include the following route:
- GET /users/:id - Responds with "User ID: [id]" where [id] is the dynamic value from the route.
- Add another route:
 - GET /products/:category/:productId - Responds with "Category: [category], Product ID: [productId]".

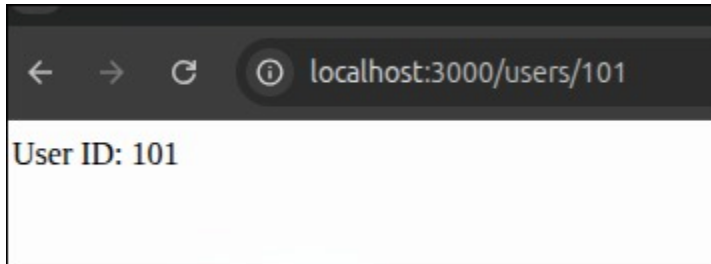
File: server2.js



```
1  const express = require('express');
2  const app = express();
3
4  app.get('/', (req, res) => {
5    res.send('Welcome to the Home Page');
6  });
7
8  app.get('/users/:id', (req, res) => {
9    const userId = req.params.id;
10   res.send(`User ID: ${userId}`);
11 });
12
13 app.get('/products/:category/:productId', (req, res) => {
14   const { category, productId } = req.params;
15
16   res.json({
17     category: category,
18     productId: productId
19   });
20 });
21
22 app.use((req, res) => {
23   res.status(404).send('Page Not Found');
24 });
25
26 app.listen(3000, () => {
27   console.log('Server running on port 3000');
28 });
```


Test URLs:

- <http://localhost:3000/users/101>



Output: User ID: 101

- <http://localhost:3000/products/electronics/555>

- **Output (JSON):**

JSON

```
{  
  "category": "electronics",  
  "productId": "555"  
}
```

